

二分查找边界与极值之美：从底层逻辑到模板化应用

二分查找的核心难点从未在于“折半搜索”的逻辑本身，而在于**边界条件的控制**（为什么有时 `right = len`，有时 `right = len - 1`？为什么有时 `left < right`，有时 `left <= right`？）。

本篇笔记将带你穿透这些表象，掌握二分查找最底层的物理规律。

第一部分：二分查找的两大核心流派

二分查找的所有边界变化，都源于你对“搜索区间”的最初定义。一旦定义好区间，后续的所有行为（循环条件、边界收缩、初始化）都必须忠实于这个契约。

流派一：左闭右开区间 `[left, right)`

- **区间物理意义：**`right` 指向的是**搜索范围外**的第一个位置。
- **适用场景：**最长递增子序列（LIS）的牌堆追加、需要返回 `len` 作为越界插入位置的场景。

模板代码（寻找第一个 $\geq target$ 的数）

```
int left = 0, right = len; // 初始右边界在有效范围外
while (left < right) {      // 当 left == right 时区间为空，终止循环
    int mid = left + (right - left) / 2; // 向下取整
    if (tails[mid] < target) {
        left = mid + 1;      // 既然 mid 处的数小于 target，目标一定在 mid 右侧，新区间
    } else {
        right = mid;         // mid 处的数可能是答案，新区间 [left, mid)
    }
}
return left; // 此时 left == right，指向第一个 >= target 的位置
```

流派二：双闭区间 `[left, right]`

- **区间物理意义：**`left` 和 `right` 都指向**确切的、有意义的有效元素下标**。
- **适用场景：**常规的值查找、寻找最后一个满足条件的元素。

模板代码（寻找第一个 $\geq target$ 的数）

```
int left = 0, right = len - 1; // 初始右边界在最后一个有效下标
while (left <= right) {        // 当 left > right 时区间才为空，终止循环
    int mid = left + (right - left) / 2;
    if (tails[mid] < target) {
        left = mid + 1;        // 新区间 [mid + 1, right]
    } else {
        right = mid - 1;       // 新区间 [left, mid - 1]
    }
}
return left; // 最终 left 依然会指向第一个 >= target 的位置
```

第二部分：深度剖析 LIS 中的极值与越界

1. 若数组中所有数都比 *target* 小，会发生什么？

在 LIS（最长递增子序列）算法中，当遇到一个比当前所有“牌堆顶”都大的新元素 x 时，我们需要将其追加到最右侧并自增 LIS 长度。

- **执行轨迹：** 由于 $\text{tails}[\text{mid}] < x$ 永远成立，指针 left 会被 $\text{left} = \text{mid} + 1$ 像推土机一样，一路向右推。
- **最终落点：** left 最终会停在 len 处（此时 $\text{left} == \text{right}$ ，循环退出）。
- **物理意义：** $\text{tails}[\text{len}] = x$ 刚好在原数组的下一个位置写入了新值，随后触发 $\text{len}++$ 。“查找”与“追加”两套逻辑在边界上完美合二为一。

2. 为什么初始化时 right 不能设为 $\text{len} - 1$ ？

如果在使用 $\text{while}(\text{left} < \text{right})$ （左闭右开）时，错误地将 right 初始化为 $\text{len} - 1$ ：

- 由于 right 被锁死在 $\text{len} - 1$ ， left 永远无法到达 len 。
- 当面对“所有数都比 *target* 小”的情况时，算法本该追加新元素，结果却会返回 $\text{len} - 1$ ，将原本的最后一个元素覆盖，导致 **LIS 长度无法增长**。



第三部分：寻找“最后一个小于 *target* 的数”的边界艺术

这个场景是二分查找中最容易写出“死循环”或“数组越界”的重灾区。

优雅方案一：对称性转化（降维打击，推荐！）

核心数学定律：

最后一个 $< \text{target}$ 的数的位置，永远在 第一个 $\geq \text{target}$ 的数的**左边相邻位置**。

利用这个规律，我们甚至不需要写新的二分查找，直接复用 LIS 的查找代码：

1. 求出第一个 $\geq \text{target}$ 的下标 idx 。
 2. $\text{idx} - 1$ 即为所求。
- **边界验证（所有数都比 *target* 小）：**
 - idx 返回 len 。
 - $\text{idx} - 1 = \text{len} - 1$ （指向数组最后一个元素）。**完全正确！**
 - **边界验证（所有数都比 *target* 大）：**
 - idx 返回 0 。
 - $\text{idx} - 1 = -1$ （代表不存在这样的数）。**完全正确！**

优雅方案二：直接二分实现（左闭右闭 + 向上取整）

如果你需要直接写出一个二分查找来定位“最后一个 $< \text{target}$ ”的数，为了让指针 left 在满足条件时能够**贪心地向右停留**（ $\text{left} = \text{mid}$ ），我们必须做出联动设计。

完美避坑实现：

```
int findLastLessThan(int *tails, int len, int target) {
    // 边界特判：若数组为空，或连最小的数都  $\geq \text{target}$ ，说明无解，直接返回 -1
    if (len <= 0 || tails[0] >= target) {
        return -1;
    }
```

```

int left = 0, right = len - 1; // 1. 必须是 len - 1! 杜绝越界
while (left < right) {
    // 2. 必须是向上取整 (+1)! 杜绝只有两个元素且 tails[left] < target 时的死循环
    int mid = left + (right - left + 1) / 2;

    if (tails[mid] < target) {
        left = mid; // 可能是当前 mid, 或者其右侧更接近 target 的数
    } else {
        right = mid - 1; // 已经 >= target 了, 答案一定在 mid 的左侧
    }
}
return left; // 此时 left == right, 一定是指向最后一个小于 target 的位置
}

```

⚡ 第四部分：防死循环与越界的“黄金配对定律”

二分查找中，mid 的取整方向必须与 left 的收缩边界保持一致，否则就会导致程序在只剩下两个元素时陷入死循环。

配对类型	中点计算 (mid)	满足条件时的 left 收缩	不满足条件时的 right 收缩	初始右边界 right 设定	物理表现
黄金配对 A (向下取整)	left + (right - left) / 2	left = mid + 1	right = mid	len	mid 永远碰不到 right，右边界安全，可为开区间
黄金配对 B (向上取整)	left + (right - left + 1) / 2	left = mid	right = mid - 1	len - 1	mid 有可能直接等于 right，右边界必须为闭区间

记忆口诀：

- 如果你在代码中写了 `left = mid`，那你的算式里就必须有 `+ 1`（向上取整），且 `right` 必须是 `len - 1`。
- 如果你在代码中写了 `right = mid`，那你的算式里就绝对不能有 `+ 1`（向下取整），且 `right` 可以是 `len`。